

Form Validation, Custom Error Pages & Login Errors — Assignment Answers

Topics: server-side validation, forward-with-error pattern, web.xml error-page, login error display

Q1. signup.jsp — Username, Email, Password Form

```
<form action="SignupServlet" method="POST">
  <input type="text" name="username" placeholder="Username" required>
  <input type="email" name="email" placeholder="Email" required>
  <input type="password" name="password" placeholder="Password (min 6 characters)" required>
  <button type="submit">Sign Up</button>
</form>
```

(Full styled file with error slots: **signup.jsp**, attached separately.)

Q2. SignupServlet — Server-Side Validation

Checks, in order: no field empty → password length ≥ 6 → email contains '@' and ends with '.com'. Any failure forwards back to signup.jsp with an error attribute set on the request.

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    String username = request.getParameter("username");
    String email = request.getParameter("email");
    String password = request.getParameter("password");

    boolean hasError = false;

    if (username == null || username.trim().isEmpty()
        || email == null || email.trim().isEmpty()
        || password == null || password.trim().isEmpty()) {
        request.setAttribute("generalError", "All fields are required.");
        hasError = true;
    }

    if (!hasError && password.length() < 6) {
        request.setAttribute("generalError", "Password must be at least 6 characters long.");
        hasError = true;
    }

    if (email != null && (!email.contains("@") || !email.toLowerCase().endsWith(".com"))) {
        request.setAttribute("emailError", "Please enter a valid email address.");
        hasError = true;
    }

    if (hasError) {
        RequestDispatcher dispatcher = request.getRequestDispatcher("signup.jsp");
        dispatcher.forward(request, response);
        return;
    }

    response.getWriter().println("<h2>Account created successfully for " + username + "!</h2>");
}
```

Q3. Showing the Email Error Below the Email Field

The servlet sets a dedicated 'emailError' request attribute (separate from the general error), and signup.jsp checks for it right below the email input to show a red inline message:

```
<input type="email" name="email" placeholder="Email" value="<%= oldEmail %>" required>
<%
    String emailError = (String) request.getAttribute("emailError");
    if (emailError != null) {
%>
        <div class="error"><%= emailError %></div>
<%
    }
%>
```

CSS used: .error { color: red; font-size: 12px; }

Q4. Custom 404 Error Page — error404.jsp via web.xml

web.xml entry:

```
<error-page>
  <error-code>404</error-code>
  <location>/error404.jsp</location>
</error-page>
```

error404.jsp:

```
<%@ page isErrorPage="true" %>
<html>
<body>
  <h1>404</h1>
  <p>Oops! The page you're looking for doesn't exist.</p>
  <a href="index.html">Back to Homepage</a>
</body>
</html>
```

To test: deploy the app and visit a non-existent URL, e.g.

<http://localhost:8080/MyWebApp/thisPageDoesNotExist> — Tomcat returns HTTP 404, and instead of the default Tomcat error page, it displays error404.jsp with the friendly message and homepage link.

Q5. LoginServlet — 'Invalid Credentials' via Request Attribute + Forward

On failed login, the servlet sets a 'loginError' request attribute and forwards (not redirects) back to login.jsp, so the error is shown in the same request/response cycle without a fresh browser-initiated page reload.

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    String username = request.getParameter("username");
    String password = request.getParameter("password");

    if (VALID_USER.equals(username) && VALID_PASS.equals(password)) {
        HttpSession session = request.getSession();
        session.setAttribute("username", username);
        response.sendRedirect("dashboard.jsp");
    } else {
        request.setAttribute("loginError", "Invalid Credentials");
        RequestDispatcher dispatcher = request.getRequestDispatcher("login.jsp");
        dispatcher.forward(request, response);
    }
}
```

login.jsp displays it above the form:

```
<%
    String loginError = (String) request.getAttribute("loginError");
    if (loginError != null) {
%>
        <p style="color:red; font-weight:bold; text-align:center;"><%= loginError %></p>
<%
    }
%>
```

Why forward, not redirect: sendRedirect() would trigger a brand-new browser request (losing the request attribute and requiring the error to be passed via a query string or session instead), whereas forward() keeps everything within the original request, letting request-scoped attributes like 'loginError' pass through cleanly to the JSP in the same server-side cycle.